Recopilación Definitiva de Programas y Ejemplos de la Biblioteca Java digitalpetri/ethernet-ip

Resumen Ejecutivo

Este informe proporciona un análisis técnico exhaustivo de la biblioteca Java digitalpetri/ethernet-ip, una implementación de cliente EtherNet/IP asíncrona y no bloqueante.[1] Se presenta una recopilación de programas y ejemplos de código anotados, cubriendo desde la conexión básica hasta casos de uso avanzados. El análisis distingue críticamente entre la interacción con dispositivos CIP genéricos (usando servicios de bajo nivel) y la comunicación con controladores Allen-Bradley Logix (usando abstracciones de *tags* simbólicos). Además, se examina el uso de esta biblioteca como una dependencia fundamental en plataformas industriales de primer nivel, como Apache PLC4X e Ignition de Inductive Automation, validando su robustez y fiabilidad para aplicaciones de misión crítica.[3]

I. Análisis de la Arquitectura de la Biblioteca digitalpetri/ethernet-ip

Fundamentos del Diseño: Asíncrono y No Bloqueante

La característica arquitectónica fundamental de la biblioteca digitalpetri/ethernet-ip es su diseño asíncrono y no bloqueante.[1] A diferencia de las implementaciones de E/S (Entrada/Salida) tradicionales que bloquean un hilo de ejecución mientras esperan una respuesta del dispositivo de campo, esta biblioteca utiliza un modelo de programación

reactivo.

Este diseño es una ventaja significativa para las aplicaciones modernas de IIoT (Internet Industrial de las Cosas) y SCADA (Supervisión, Control y Adquisición de Datos). Permite que una aplicación maneje un gran número de conexiones y solicitudes simultáneas con un número mínimo de hilos, mejorando drásticamente la escalabilidad y el rendimiento. En un sistema de monitoreo de alta velocidad, un hilo de E/S bloqueado podría degradar la capacidad de respuesta de toda la aplicación; este diseño evita ese escenario.

Disección de Módulos (El Artefacto com.digitalpetri.enip)

La biblioteca no es un artefacto monolítico. El análisis de los repositorios de artefactos de Maven [6] y la estructura del repositorio de código fuente [1] revela una arquitectura multi-módulo clara. Comprender la distinción entre estos módulos es esencial para utilizar la biblioteca correctamente.

Módulos Centrales (-core):
Estos módulos forman la base del protocolo y no suelen ser utilizados directamente por las aplicaciones cliente.

- enip-core: Proporciona los *codecs* (codificadores/decodificadores) y las estructuras de datos para el protocolo EtherNet/IP (E/IP) en sí. Este es el protocolo de encapsulación que opera sobre TCP (generalmente en el puerto 44818) y UDP.[7]
- cip-core: Proporciona las estructuras de datos para el *Common Industrial Protocol* (CIP).[6] CIP es el protocolo de capa de aplicación orientado a objetos que se transporta *dentro* de los paquetes E/IP.[8] Define el modelo de objetos de Clase, Instancia y Atributo que utilizan todos los dispositivos compatibles.

Módulos de Cliente (-client):
Estos son los puntos de entrada para los desarrolladores.

- enip-client: Un cliente de nivel de encapsulación.[6] Gestiona la sesión de E/IP con el dispositivo y es responsable de servicios básicos como ListIdentity.
- cip-client: Construido sobre enip-client, este módulo implementa la lógica para enviar y recibir servicios CIP.[6] Es el cliente principal para interactuar con dispositivos CIP genéricos mediante la invocación de servicios explícitos (conocidos como mensajería de Clase 3 [9]), como GetAttributeSingleService.
- logix-services: Un módulo de abstracción de alto nivel.[6] Este cliente especializado entiende los servicios específicos de los controladores de la familia Logix de Allen-Bradley (ej. ControlLogix, CompactLogix). Proporciona la capacidad de interactuar directamente con *tags* simbólicos por su nombre, en lugar de direcciones de bajo nivel.[3]

La Bifurcación Crítica: Logix vs. CIP Genérico

La estructura de módulos de la biblioteca expone una bifurcación de diseño fundamental que un desarrollador debe abordar: ¿El dispositivo de destino es un controlador de la familia Logix o es un dispositivo CIP genérico?

La confusión es un tema recurrente en las discusiones de la comunidad. Los desarrolladores provenientes de otras bibliotecas, como pycomm en Python, esperan una simple función read_tag("N7:54").[3] Sin embargo, los ejemplos básicos de CipClient [1] requieren un conocimiento de bajo nivel de los IDs de Clase, Instancia y Atributo [3], lo cual resulta confuso.

La solución a esta confusión radica en la selección del módulo correcto:

1. **Dispositivos CIP Genéricos:** (p.ej., variadores de frecuencia, módulos de E/S de terceros [9], sensores, robots). Se *debe* utilizar el cip-client. El desarrollador es responsable de obtener el modelo de objetos del dispositivo (generalmente desde un archivo EDS o un manual del fabricante) para saber qué ruta de Clase/Instancia/Atributo consultar.[9]
2. **Controladores Allen-Bradley Logix:** Se *debe* utilizar logix-services. Este módulo abstrae la complejidad del protocolo Logix y permite la lectura y escritura de *tags* por su nombre simbólico [3], que es lo que la mayoría de los usuarios de Allen-Bradley esperan.

La siguiente tabla resume esta decisión de arquitectura.

Tabla 1: Matriz de Decisión de Módulos de Cliente digitalpetri/ethernet-ip

Módulo (Artefacto Maven)	Nivel de Abstracción	Dispositivos de Destino	Caso de Uso Principal
enip-client	Bajo (Nivel de Encapsulación)	Cualquier dispositivo E/IP	Gestión de sesión, ListIdentity
cip-client	Medio (Nivel de Servicio CIP)	Dispositivos CIP Genéricos	GetAttributeSingle, SetAttributeSingle
logix-services	Alto (Nivel de *Tag* Simbólico)	Familia Allen-Bradley Logix	readTag("MyTagName"), writeTag("MyTagNa

			me")

II. Configuración Inicial y Ejemplos de "Quick Start" (Análisis del README)

Los ejemplos de "Quick Start" proporcionados en el README.md del repositorio [1] sirven como los programas canónicos de "hola mundo" para la biblioteca.

Ejemplo 1: El EtherNetIpClient (Descubrimiento de Identidad)

Este programa demuestra el uso del cliente de nivel más bajo (EtherNetIpClient) para ejecutar un servicio de E/IP (no CIP) llamado ListIdentity.

Código de Ejemplo (Anotado) [1]:

Java

```
// 1. Configuración usando el patrón builder
// Se especifica la IP del dispositivo y otros parámetros (VendorId, SerialNumber, Timeout).
EtherNetIpClientConfig config = EtherNetIpClientConfig.builder("10.20.4.57")
  .setSerialNumber(0x00)
  .setVendorId(0x00)
  .setTimeout(Duration.ofSeconds(2))
  .build();

// 2. Instanciación del cliente de nivel de red
EtherNetIpClient client = new EtherNetIpClient(config);

// 3. Conexión asíncrona. Se usa.get() para bloquear el hilo
// actual y esperar a que la conexión se complete (solo para este ejemplo).
client.connect().get();
```

```
// 4. Invocación del servicio asíncrono ListIdentity.
// whenComplete adjunta un callback que se ejecutará cuando la respuesta (li) o la excepción (ex) esté disponible.
client.listIdentity().whenComplete((li, ex) -> {
    if (li!= null) {
        // 5. Impresión de los atributos del Objeto Identidad
        // El Objeto Identidad es estándar en todos los dispositivos E/IP.
        li.getIdentity().ifPresent(id -> {
            System.out.println("productName=" + id.getProductName());
            System.out.println("revisionMajor=" + id.getRevisionMajor());
            System.out.println("revisionMinor=" + id.getRevisionMinor());
        });
    } else {
        // 6. Manejo de excepciones
        ex.printStackTrace();
    }
});

// 7. Desconexión (también asíncrona)
client.disconnect().get();

// 8. Liberación de recursos compartidos (CRÍTICO)
// Esto cierra los pools de hilos de Netty y otros recursos.
EtherNetIpShared.releaseSharedResources();
```

Análisis y Propósito:
El propósito de este programa no es leer datos de proceso. Su función es actuar como un "ping" de EtherNet/IP para el descubrimiento de activos y la verificación de la conectividad. La llamada a EtherNetIpShared.releaseSharedResources() es crítica 1; omitirla en una aplicación de larga duración o en pruebas repetidas puede llevar a fugas de recursos (específicamente, los pools de hilos de Netty).

Ejemplo 2: El CipClient (Lectura de Atributos CIP Genéricos)

Este programa es más avanzado y demuestra una interacción CIP explícita usando el CipClient para leer atributos del Objeto Identidad del dispositivo.

Código de Ejemplo (Anotado) [1]**:**

Java

```java
// 1. Configuración (idéntica al Ejemplo 1)
EtherNetIpClientConfig config = EtherNetIpClientConfig.builder("10.20.4.57")
    //... configuraciones...
  .build();

// 2. Definición de la Ruta de Conexión (Connection Path)
// Esto le dice al gateway E/IP cómo enrutar el mensaje CIP.
// new PortSegment(1, new byte{(byte) 0}) significa:
// Puerto 1 (el backplane) y luego Slot 0.
PaddedEPath connectionPath = new PaddedEPath(
    new PortSegment(1, new byte{(byte) 0})
);

// 3. Instanciación del cliente de nivel de protocolo CIP
// Note que se pasa tanto la configuración como la ruta de conexión.
CipClient client = new CipClient(config, connectionPath);

client.connect().get();

// 4. Definición del Servicio CIP a invocar
// GetAttributeListService: Leer una lista de atributos
GetAttributeListService service = new GetAttributeListService(
    // 5. Ruta del Objeto: Clase 0x01 (Identity Object), Instancia 0x01
    new PaddedEPath(new ClassId(0x01), new InstanceId(0x01)),
    new int{4}, // Solicitando el Atributo 4 (Revisión)
    new int{2}  // Solicitando el Atributo 2 (Número de Serie)
);

// 6. Invocación del servicio (en este caso, sin conexión CIP)
client.invokeUnconnected(service).whenComplete((as, ex) -> {
    if (as!= null) {
        try {
            // 7. Procesamiento del ByteBuf crudo
            // La respuesta es un array de datos de atributos.
            ByteBuf data = as.getData();
            int major = data.readUnsignedByte();
            int minor = data.readUnsignedByte();
            System.out.println(String.format("firmware v%s.%s", major, minor));
        } catch (Throwable t) {
            t.printStackTrace();
```

```
        } finally {
            // 8. Liberación de los buffers de respuesta
            Arrays.stream(as).forEach(a -> ReferenceCountUtil.release(a.getData()));
        }
    } else {
        ex.printStackTrace();
    }
});

// 9. Desconexión y liberación de recursos
client.disconnect().get();
EtherNetIpShared.releaseSharedResources();
```

Análisis y Propósito:
Este ejemplo es la base para toda la mensajería CIP explícita (Clase 3). Demuestra que un desarrollador debe definir dos rutas distintas:

1. La connectionPath: La ruta de red/chasis para *alcanzar* el procesador (p.ej., "backplane, slot 0").
2. La PaddedEPath del servicio: La ruta de objeto *dentro* del procesador (p.ej., "Clase 1, Instancia 1, Atributo 4").

Este programa lee metadatos del dispositivo (versión de firmware), no *tags* de proceso. Sin embargo, el patrón que establece es exactamente el que se debe expandir para leer datos de E/S de dispositivos genéricos.

III. Interacción con Dispositivos CIP Genéricos (El Enfoque de Bajo Nivel)

Esta sección aborda cómo interactuar con la gran mayoría de dispositivos EtherNet/IP que *no* son PLCs Logix, como variadores de frecuencia, módulos de E/S, sensores y robots.

Contexto: El Modelo de Objetos CIP y los Archivos EDS

El *Common Industrial Protocol* (CIP) es fundamentalmente un protocolo orientado a objetos.[8] Toda la funcionalidad de un dispositivo se expone a través de un modelo de datos que

consiste en Clases (Objetos), Instancias y Atributos.

Para leer "datos de proceso" de un dispositivo genérico, no se solicita un "tag" simbólico, ya que este concepto es una abstracción específica de Logix. En su lugar, se solicita un atributo específico (generalmente el atributo de "datos") de una instancia del "Objeto de Ensamblaje" (Assembly Object).

La información sobre qué Clase, Instancia y Atributo utilizar para E/S (Entrada/Salida) no es estándar en todos los dispositivos y debe obtenerse del manual del fabricante del dispositivo o de su archivo **EDS (Electronic Data Sheet)**.[9]

Programa de Ejemplo: Lectura de Registros de E/S con GetAttributeSingleService

Este programa de ejemplo se sintetiza a partir de una solución técnica proporcionada en el rastreador de *issues* (incidencias) de GitHub del proyecto.[9]

Caso de Uso: Un desarrollador necesita leer el estado de las entradas digitales de un módulo de E/S genérico [9], una tarea que anteriormente realizaba usando el protocolo Modbus.

Código de Ejemplo (Anotado) [9]:

Java

```
// Asumimos que 'client' es un CipClient configurado y conectado
// como en el Ejemplo 2 de la Sección II.
// CipClient client =...;
// client.connect().get();

// 1. Definición de la Ruta de Atributos Específica
// Como sugiere el autor de la biblioteca , los dispositivos de E/S
// genéricos comúnmente usan el Objeto de Ensamblaje (Clase 0x04).
// - Clase 0x04: Assembly Object
// - Instancia 103: Típicamente usada para Entradas (Inputs)
// - Atributo 3: Típicamente el atributo de Datos
PaddedEPath requestPath = new PaddedEPath(
    new LogicalSegment.ClassId(0x04),
    new LogicalSegment.InstanceId(103),
```

```
    new LogicalSegment.AttributeId(3)
);

// 2. Creación del servicio CIP
// GetAttributeSingleService es el servicio estándar para leer un único atributo.
GetAttributeSingleService service = new GetAttributeSingleService(requestPath);

// 3. Invocación del servicio y obtención de la respuesta
//.get() se usa para la simplicidad del ejemplo.
ByteBuf data = client.invoke(service).get();

// 4. Procesamiento de los datos crudos
// 'data' es un buffer de bytes de Netty (ByteBuf) que contiene los
// bytes crudos del registro de entrada.
System.out.println("Datos de E/S crudos (hex): " + ByteBufUtil.hexDump(data));

// 5. El desarrollador es ahora responsable de decodificar este ByteBuf
// (p.ej., data.readBoolean(), data.readUnsignedShort()) según el
// mapa de memoria definido por el fabricante del dispositivo.

// 6. Liberar el ByteBuf (¡Muy importante!)
// Los ByteBufs de Netty son contados por referencia. No liberarlos causa fugas de memoria.
ReferenceCountUtil.release(data);

//... al finalizar, desconectar y liberar recursos compartidos...
// client.disconnect().get();
// EtherNetIpShared.releaseSharedResources();
```

Análisis y Propósito:
Este programa es la solución directa al problema de "No sé los IDs".3 La respuesta es que los IDs son específicos del fabricante y deben buscarse en la documentación.
La dificultad que experimentan los usuarios [3] no se debe a una deficiencia en digitalpetri/ethernet-ip, sino que es una consecuencia inherente del diseño del protocolo CIP [8] para dispositivos genéricos. Este ejemplo [9] es el programa de lectura de E/S más importante para cualquier dispositivo que no sea Logix.

Consideraciones para la Escritura de Atributos (SetAttributeSingleService)

Aunque los materiales de investigación no proporcionan un ejemplo explícito de escritura

genérica [16], el patrón es simétrico al de lectura:

1. Se construiría una PaddedEPath apuntando al ensamblaje de salida (p.ej., LogicalSegment.InstanceId(104) como se sugiere en [9]).
2. Se crearía un ByteBuf y se rellenaría con los bytes de datos a escribir.
3. Se instanciaría un SetAttributeSingleService, pasando tanto la ruta como el ByteBuf de datos.
4. Se invocaría el servicio: client.invoke(service).get().
5. Se liberaría el ByteBuf de solicitud después de que la invocación se complete.

IV. Interacción con PLCs Logix (El Enfoque de Alto Nivel)

Esta sección se enfoca en el programa que la mayoría de los desarrolladores que trabajan con controladores Allen-Bradley (ControlLogix, CompactLogix) desean: leer y escribir *tags* por su nombre simbólico.

El Módulo logix-services: Resolviendo la Brecha de Abstracción

El módulo logix-services [6] es la solución a la brecha de abstracción identificada en [3] y.[3] Resuelve la frustración del desarrollador que está atascado con IDs de bajo nivel. El propio autor de la biblioteca, kevinherron, confirmó esto en una discusión pública [3], declarando: "Pruebe el ejemplo en el módulo logix-services... Muestra la lectura de un *tag* direccionado por su nombre simbólico".

Este módulo implementa los servicios PCCC/Logix específicos (como ReadTagService y WriteTagService) que entienden los nombres de *tags*, manejando toda la resolución de direcciones de bajo nivel internamente.

Programa de Ejemplo: Lectura y Escritura de *Tags* Logix

Este programa se sintetiza basándose en la recomendación del autor [3] y la referencia en el README [1] al "Logix Example". Demuestra la simplicidad que se obtiene al usar el cliente de

alto nivel correcto.

Código de Ejemplo (Sintetizado y Anotado):

Java

```java
// Asumimos las importaciones de com.digitalpetri.enip.logix.*
// y com.digitalpetri.enip.cip.datatypes.*

// 1. Configuración (similar al Ejemplo 1 de la Sección II)
EtherNetIpClientConfig config = EtherNetIpClientConfig.builder("10.20.4.57")
  .build();

// 2. Definición de la Ruta de Conexión (Backplane, Slot 0)
PaddedEPath connectionPath = new PaddedEPath(
    new PortSegment(1, new byte{(byte) 0})
);

// 3. Instanciación del cliente Logix
// ¡Note el uso de LogixClient, no CipClient!
LogixClient client = new LogixClient(config, connectionPath);

// 4. Conexión
client.connect().get();

// 5. LECTURA DE TAG (POR NOMBRE)
// Se especifica el nombre del tag y el tipo de datos Java esperado.
// La biblioteca maneja la deserialización.
client.readTag("MyDintTag", DINT.class).whenComplete((value, ex) -> {
    if (value!= null) {
        // 'value' ya es un tipo de datos Java nativo (ej. Short para DINT)
        System.out.println("Valor del tag 'MyDintTag': " + value);
    } else {
        ex.printStackTrace();
    }
});

// 6. ESCRITURA DE TAG (POR NOMBRE)
// Se pasa el nombre del tag y el valor Java nativo a escribir.
// La biblioteca maneja la serialización.
client.writeTag("MyDintTag", (short) 1234).whenComplete((v, ex) -> {
```

```
    if (ex == null) {
        System.out.println("Escritura en 'MyDintTag' completada.");
    } else {
        ex.printStackTrace();
    }
});

//... esperar a que se completen las operaciones asíncronas...

// 7. Desconexión y liberación
client.disconnect().get();
EtherNetIpShared.releaseSharedResources();
```

Análisis y Propósito:
Este programa es la respuesta directa a la frustración del desarrollador de 3/.3 Al usar LogixClient, el paradigma de programación cambia de un modelo de "Clase/Instancia/Atributo" a uno de "Nombre/Valor", que es mucho más intuitivo para los programadores de PLC.

Funcionalidad Avanzada: Descubrimiento de *Tags* (Tag Browsing)

Una capacidad avanzada requerida por muchas aplicaciones HMI/SCADA es el "tag browsing", o la capacidad de listar programáticamente todos los *tags* disponibles en un controlador.

La *Issue* #8 de GitHub [10] pregunta específicamente por esta capacidad. La incidencia fue cerrada por el autor de la biblioteca con el estado "completado".[10] La discusión técnica dentro de esa incidencia revela cómo se logra esto:

Un programa de descubrimiento de *tags* utiliza servicios CIP como GetInstanceAttributeListService en un objeto específico de Logix conocido como "Symbol Object". Al iterar sobre las instancias de este objeto, el programa puede leer sus atributos, que incluyen:

- Atributo 1: El nombre del *tag* (como un String).
- Atributo 2: El tipo de datos del *tag*.
- Atributo 8: Las dimensiones del *tag* (si es un array).[10]

El módulo logix-services probablemente envuelve esta compleja lógica de iteración en una API más simple, permitiendo a una aplicación construir un árbol de *tags* del PLC dinámicamente en tiempo de ejecución.

V. Programas del Ecosistema que Utilizan digitalpetri/ethernet-ip

El valor y la robustez de una biblioteca de código abierto pueden medirse por los proyectos que dependen de ella. Esta sección analiza dos programas/plataformas clave del ecosistema de automatización que utilizan digitalpetri/ethernet-ip como un componente fundamental.

Caso de Estudio 1: Apache PLC4X

Apache PLC4X es un proyecto de la Apache Software Foundation (ASF) cuyo objetivo es proporcionar un conjunto unificado de bibliotecas para comunicarse con PLCs de varios fabricantes.[11]

- **Evidencia de Uso:** Múltiples fuentes confirman que PLC4X se "construye sobre" digitalpetri/ethernet-ip para su *driver* EtherNet/IP.[3] El *driver* plc4j-driver-eip [12] enumera a com.digitalpetri.enip como una dependencia.[14]
- **Análisis de Abstracción:** digitalpetri/ethernet-ip actúa como el "motor" de protocolo (la implementación de E/IP/CIP), mientras que PLC4X proporciona una "transmisión" (una API unificada). PLC4X añade su propia capa de abstracción, incluyendo un formato de dirección simplificado como %{tagname} [13], que su *driver* traduce internamente en las llamadas apropiadas a logix-services o cip-client.
- **Implicaciones:** La elección de esta biblioteca por parte de un proyecto de la ASF es un fuerte respaldo a su corrección técnica, calidad de código y conformidad con los estándares.

Caso de Estudio 2: Ignition (Inductive Automation)

Ignition, desarrollado por Inductive Automation, es una de las plataformas SCADA/IIoT comerciales más importantes del mercado.

- **Evidencia de Uso:** Un análisis de dependencias (usando jdeps) del módulo del *driver* Logix comercial de Ignition, compartido en una discusión pública [4], revela dependencias directas e inequívocas de los paquetes de la biblioteca: com.digitalpetri.enip,

com.digitalpetri.enip.cip, com.digitalpetri.enip.cip.epath, etc.
- **Análisis de Causalidad:** La conexión es aún más profunda. El principal contribuyente y mantenedor de la biblioteca de código abierto digitalpetri/ethernet-ip (cuyo nombre de usuario es kevinherron [3]) es también un empleado de Inductive Automation.[4]

Implicaciones:
Esta conexión es la validación industrial definitiva. La biblioteca no es simplemente un proyecto de hobby "utilizado por" una empresa comercial; es un componente central de una plataforma industrial de misión crítica, desarrollado y mantenido por la misma organización. Esto significa que la biblioteca digitalpetri/ethernet-ip es:

1. **Robusta:** Ha sido probada en campo en miles de instalaciones industriales.
2. **Mantenida:** Es mantenida activamente por expertos pagados cuyo trabajo depende de su fiabilidad.
3. **Fiable:** Se puede considerar la opción menos riesgosa y el estándar *de facto* para cualquier proyecto profesional de Java que requiera comunicación EtherNet/IP.

VI. Guía de Campo: Diagnóstico de Problemas y Patrones de Solución

Una recopilación de programas está incompleta sin una guía para depurarlos. Un análisis de los foros de *Discussions* (Discusiones) de GitHub [15] revela patrones de problemas comunes que enfrentan los desarrolladores.

Problema 1: java.util.concurrent.TimeoutException

- **Descripción:** La aplicación lanza una excepción de tiempo de espera, como la reportada en.[15]
- **Análisis:** Esto ocurre cuando la biblioteca no recibe una respuesta del dispositivo de campo dentro del período especificado en EtherNetIpClientConfig.[1]
- **Causas Probables y Soluciones:**
  1. **Red Lenta o Congestionada:** La latencia de la red excede el tiempo de espera. Solución: Aumentar el valor de .setTimeout(Duration.ofSeconds(X)) en el EtherNetIpClientConfig.
  2. **Dispositivo Sobrecargado:** El PLC está ocupado (alta utilización de la CPU) y no puede responder a solicitudes de mensajería explícita a tiempo.

3. **Firewall:** Un *firewall* está bloqueando el tráfico. El puerto TCP 44818 (y el puerto UDP 44818 para *broadcasts*) debe estar abierto en ambas direcciones.[7]

Problema 2: ConnectionFailure for AllenBradley... with Firmware version 36

- **Descripción:** Un fallo de conexión que parece estar vinculado a una versión específica del *firmware* del dispositivo.[15]
- **Análisis:** Esto resalta un riesgo clave de la comunicación con protocolos propietarios. El fabricante (Rockwell Automation) puede introducir cambios en las actualizaciones de *firmware* que rompan la compatibilidad con implementaciones de terceros.
- **Solución:** Consultar activamente las *issues* y *discussions* del repositorio [5] para parches o soluciones alternativas. Es crucial probar la aplicación contra las versiones de *firmware* exactas de los dispositivos de producción.

Problema 3: Can not read attribute (No se puede leer el atributo)

- **Descripción:** El dispositivo devuelve un error de CIP que indica que el atributo no se puede leer.[15]
- **Análisis:** Este es casi siempre un caso de "confusión de paradigma" (Logix vs. CIP Genérico). El análisis de la discusión 79 de GitHub [15] indica que el usuario probablemente estaba usando un servicio de alto nivel (como ReadTagService de logix-services) en un dispositivo que *no* era Logix y no entendía ese servicio.
- **Solución:** Volver a los fundamentos de la Sección III. Usar el CipClient genérico y el servicio GetAttributeSingleService [9] con la ruta de Clase/Instancia/Atributo correcta, obtenida del archivo EDS o del manual del dispositivo.

Problema 4: When to release the connection and when not (Cuándo liberar la conexión y cuándo no)

- **Descripción:** Una pregunta de la comunidad [15] sobre la gestión del ciclo de vida de la conexión.
- **Patrones de Solución:**

1. **Aplicaciones de "Una Sola Vez" (Scripts, Pruebas):** El ciclo de vida es corto. El patrón debe ser: connect() -> invoke() -> disconnect() -> EtherNetIpShared.releaseSharedResources().[1]
2. **Aplicaciones de Larga Duración (Servicios, HMI/SCADA):** El cliente debe ser un objeto de larga vida. El patrón es: connect() *una vez* al inicio de la aplicación. Mantener la conexión (la biblioteca manejará las reconexiones). Llamar a disconnect() y EtherNetIpShared.releaseSharedResources() *solo* durante el apagado de la aplicación.

VII. Conclusión y Recomendaciones de Implementación

Síntesis

La biblioteca digitalpetri/ethernet-ip es la implementación de cliente EtherNet/IP de facto y de grado industrial para el ecosistema Java. Su diseño asíncrono y no bloqueante la hace adecuada para aplicaciones de alto rendimiento. Su adopción como componente central en plataformas líderes como Apache PLC4X [3] e Ignition [4] proporciona una validación inequívoca de su robustez y fiabilidad para aplicaciones de misión crítica.

Recomendaciones de Implementación (Mejores Prácticas)

Basado en el análisis de sus programas, arquitectura y problemas comunes, se recomiendan las siguientes mejores prácticas:

1. **Práctica 1: La Decisión Inicial (Selección de Módulo).** La decisión de arquitectura más crítica es la selección del cliente.
   - **Use logix-services** si su dispositivo de destino es un controlador Allen-Bradley de la familia Logix. Esto permite la interacción intuitiva de lectura/escritura de *tags* por nombre simbólico.[3]
   - **Use cip-client** para todos los demás dispositivos CIP genéricos (variadores, E/S de terceros, etc.). Esté preparado para consultar los manuales del fabricante o los archivos EDS para determinar las rutas de Clase/Instancia/Atributo correctas para

E/S.[9]

2. **Práctica 2: Gestión Rigurosa de Recursos.** La naturaleza asíncrona y de bajo nivel de la biblioteca exige una gestión cuidadosa de los recursos.
   - Maneje los CompletableFuture devueltos por la biblioteca correctamente, adjuntando *callbacks* de éxito y fracaso.
   - Recuerde que los búferes de datos (como el ByteBuf devuelto por GetAttributeSingleService [9]) son contados por referencia. **Siempre debe liberarlos** (p.ej., ReferenceCountUtil.release(data)) después de su uso para evitar fugas de memoria.
   - **Siempre llame a EtherNetIpShared.releaseSharedResources()** [1] al cerrar su aplicación para liberar los *pools* de hilos de Netty.
3. **Práctica 3: Enfoque de Depuración.** Al diagnosticar problemas, asuma que la biblioteca es correcta y que el problema reside en el entorno o en las suposiciones.
   - Verifique la red: ¿Está el puerto TCP 44818 abierto?[7] ¿Hay alta latencia?
   - Verifique el dispositivo: ¿Está el PLC en modo "Run"? ¿Está sobrecargado? ¿Es la versión de *firmware* compatible?[15]
   - Verifique la suposición: ¿Está usando logix-services para leer un dispositivo genérico? (Ver Can not read attribute [15]). ¿Está usando cip-client para leer un *tag* Logix por nombre? (No funcionará).

Obras citadas

1. Asynchronous, non-blocking, EtherNet/IP client implementation for Java - GitHub, fecha de acceso: noviembre 14, 2025, https://github.com/digitalpetri/ethernet-ip
2. digitalpetri - GitHub, fecha de acceso: noviembre 14, 2025, https://github.com/digitalpetri
3. Using CIP from Java : r/PLC - Reddit, fecha de acceso: noviembre 14, 2025, https://www.reddit.com/r/PLC/comments/drjnvh/using_cip_from_java/
4. Moving away from Rockwell Automation, but want to continue using Ignition. Any suggestions? : r/PLC - Reddit, fecha de acceso: noviembre 14, 2025, https://www.reddit.com/r/PLC/comments/z4goou/moving_away_from_rockwell_automation_but_want_to/
5. Issues · digitalpetri/ethernet-ip - GitHub, fecha de acceso: noviembre 14, 2025, https://github.com/digitalpetri/ethernet-ip/issues
6. com.digitalpetri.enip - Maven Repository, fecha de acceso: noviembre 14, 2025, https://mvnrepository.com/artifact/com.digitalpetri.enip
7. This tutorial will show you, how to establish communication - Ethernet/IP in practice, fecha de acceso: noviembre 14, 2025, https://d2000.ipesoft.com/blog/how-to-establish-communication-ethernet-ip-in-practice/
8. EtherNet/IP - Wikipedia, fecha de acceso: noviembre 14, 2025, https://en.wikipedia.org/wiki/EtherNet/IP
9. Example for register read and write? · Issue #34 · digitalpetri/ethernet-ip - GitHub,

fecha de acceso: noviembre 14, 2025, https://github.com/digitalpetri/ethernet-ip/issues/34
10. Browse tags support · Issue #8 · digitalpetri/ethernet-ip - GitHub, fecha de acceso: noviembre 14, 2025, https://github.com/digitalpetri/ethernet-ip/issues/8
11. apache/plc4x: PLC4X The Industrial IoT adapter - GitHub, fecha de acceso: noviembre 14, 2025, https://github.com/apache/plc4x
12. plc4j-driver-eip - org.apache.plc4x - Maven Repository, fecha de acceso: noviembre 14, 2025, https://mvnrepository.com/artifact/org.apache.plc4x/plc4j-driver-eip
13. EtherNet/IP - Apache PLC4X, fecha de acceso: noviembre 14, 2025, https://plc4x.apache.org/plc4x/latest/users/protocols/eip.html
14. com.digitalpetri.enip » enip-client (Usages) - Maven Repository, fecha de acceso: noviembre 14, 2025, https://mvnrepository.com/artifact/com.digitalpetri.enip/enip-client/usages
15. digitalpetri ethernet-ip · Discussions · GitHub, fecha de acceso: noviembre 14, 2025, https://github.com/digitalpetri/ethernet-ip/discussions
16. EthernetIP Tag Write VI - NI - National Instruments, fecha de acceso: noviembre 14, 2025, https://www.ni.com/docs/en-US/bundle/ni-industrial-communications-ethernetip/page/ethernetip/ethernetip_tag_write.html
17. rseip - crates.io: Rust Package Registry, fecha de acceso: noviembre 14, 2025, https://crates.io/crates/rseip/0.1.0